5

Chapter 5 – Participating in open source development

On Chapter 4 you became acquainted with the software development process for OSS, and the relationship between the different groups in the community. This chapter is related to Chapter 4; however, the focus is more on the communication and social aspects when participating in an open source development project.

In this chapter you will learn about:

- How different communities operate, and how you can fit in
- Different channels for effective communication across the community
- How you should interact with others contributors and users

Note:

The writing style starting from this chapter is somewhat different than in the previous chapters. This is done on purpose. The writing style is used to also reflect the way many people in OSS communities communicate.

5.1 Participating in open source development: The big picture

Open source development is quickly gaining traction as one of the most effective development models. Just imagine the consequences of a having a handful of techies and savvy eyes going through your code, constantly reviewing it, correcting and adding to it. In little to no time your project would reach a level of quality you couldn't ever have imagined.

As discussed in *Chapter 4*, every major open source project has its own model of organization, a tight thought hierarchy where different people with different functions and different skills work in conjunction, most commonly across the Web. *Figure 5.1* illustrates a traditional organization of an open source community.

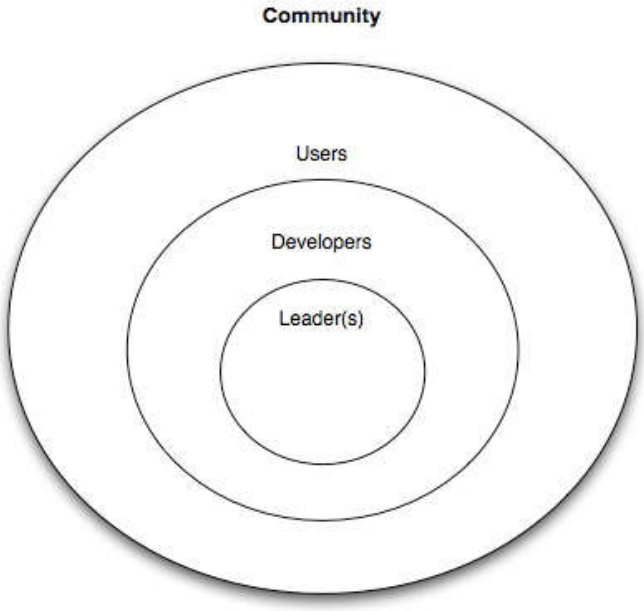


Figure 5.1: Traditional organization of an open source community

In *Chapter 4*, the focus was mainly on the groups responsible for the development of the software. In this chapter, we are taking a broader view. Notice how users take part in the organization as opposed to the closed software development method where users cannot take a more direct participation.

In order to contribute to an open source project a common misconception is that you have to be some kind of hacker; however, this is not the case. There are many ways one can contribute to an open source project.

Positions on a project vary according to project dimension; some projects behave much like a company. In those cases projects are divided into departments such as, development, public relationships, documentation, etc.; the structure reflects the project needs. What you should keep in mind is that there are many positions to occupy. In contrast, small projects are typically composed of a main developer with several co-developers. Though you are likely to be assigned your role when you first start contributing to a project, here is a description of the different roles normally available:

- **Project Leader / Main developer**: A leader performs functions that are not too different from a company's CEO: He is the one making tough decisions, and in big projects he is also in charge of coordinating many of the project subdivisions. Smaller projects often are governed by several developers, similar to a company's board of directors.

- **Programmer**: There aren't as many programmers as one would think; in many active projects the programmers are a small percentage of all contributors. The programmer is in charge of implementing features and fixing bugs.
- **Documentation Writer**: In the past open source software was difficult to install and only those who had the patience and skill could get this software working. This was perhaps one of the reasons why documentation became very important, as well as the fact that community members often rotate software development positions. It is the responsibility of the documentation writer to ensure that every person can use the software in hand.
- **Translator**: Even though English is the official Web language, it is true that not all of us are fluent in it, for that same reason it is common for a project to readily make its manuals available in as many languages as possible. Any user is more than welcome to translate any piece of the documentation to his own native language.
- **Designer**: Gone are the days when people in the Free and Open Source Software community didn't believe in image and its overall importance to a project. There are many kinds of designers. UI designers often called usability engineers, Web designers, and artwork designers. Projects like KDE or Gnome rely heavily on designers work.
- **Public relationship**: In some cases, such is the dimension of a project that huge amount of content is generated every day. Keeping the community up-to-date can be an overwhelming job. In order to keep a coherent image, a public relations team is assigned to key areas, passing the message along through the community communication channels.
- **Active User**: Active users of the software are the base of the pyramid. They use the software, provide feedback, and may customize the product to their needs.

The strength of the open source development model comes from the user base and the power given to it. A healthy project will empower users to submit feature requests, fill out bug reports, and will welcome user cooperation using properly administered communication channels. In a later section you will be introduced to some of the most used means of communication within a community, and you will learn effective ways of getting the word across.

5.2 Open source communities

Chances are you have found a certain software tool that satisfies your needs, but you don't have the money to pay for it. So you search through the Internet for an alternative free version, and after some time you find a free open source software that seems to have all that you need. Quickly you go through the features list and bummer!, an important feature that would be useful to you is missing. Most times someone with similar needs would have already gone through that same path and contributed with code that would have provided this specific feature; however, not this time; this time it's up to you!.

Contributing with code for the desired feature may take you weeks, but in the process you will learn tones of new and interesting information, gain a few ideas that would make that software snappier and meet people from all over the world who share the same passion you do; like the one of sitting alone at night with a stash of chips and a can of coke, cracking a problem like a mad man in pajamas, waiting for the moment the solution comes down that stairway from heaven, and rejoicing when it arrives by dancing with his cat.

Open source communities are full of people like this, well, maybe not all of them dance with their cats, but they love to be challenged and aren't afraid to get involved. For them opportunity often appears from necessity, which generates motivation which then mutates into fun and responsibility.

If you are reading this book, we assume you are interested in open source, and if you haven't already contributed in some way, maybe it's because you don't know where or how to start. Well, as many things in life, you have to find the need first; motivation is just not enough. By "need" we mean that you should search for a project you have a special interest in, for instance, start by looking for the ones you use on a day-to-day basis.

Take GNU/Linux for example. Many of us in the open source community use GNU/Linux daily, for it is a very robust, very powerful operating system, and with its many distributions it makes for a very good playground for people to enter the FLOSS community. Debian is one of those distributions, one of the oldest as a matter of fact, and it drives itself from a strict belief in free software as depicted in their "social contract", which guarantees you that in a bare installation, Debian contains only free software. Of the thousands of applications that make up Debian or almost any other distribution, many have their own community. Debian itself has a community.

It is then a matter of time that in your daily routine taming around your computer, you strike a bug in one of the thousands of software that make up Debian. Impaired and with no other alternative to continue whatever is you were doing, you look for a way to solve it. The bug hunting leads you to a little library with a very simple mission, intrigued you want to know more about that little library. Fortunately it is not hard in the open source world to find good documentation even to the simplest piece of software; good documentation guarantees project continuity and enables community feedback. Even small libraries like this one that is bugging you may have a community attached. It comes to no surprise, libraries are used by bigger applications, and the developers of these applications rely on these libraries for very specific functionalities; it's a mutual interest. If you are lucky, you may even find a bug tracking tool. For some of the smaller software projects that have no need for a bug tracking tool, the project leader normally goes with a simpler and centralized feedback platform, like a forum. It's very hard to be the first to find a bug in a community as large as Debian's, rest assure that if this example were to suit reality someone would have already fill in the bug for you. But in this reality you are the master, checking out every bug that bugs you. A little debugging with a proper software tool, a simple run through the source code and there it was. Five minutes later you have set up a patch, you go into the bug tracking tool and submit it, a few hours later one of the developers picks it up, merges it into the main code and redistributes it. Your good deed for the day is done.

Now don't get the wrong idea, it is not all dull work and necessity that makes an open source contributor. Some of us, specially the younger ones, have a tendency to get bored quickly. There's a little noise bubbling in our heads, like kids on Christmas Eve that just have a lot of curiosity. We go to where our current obsession drives us.

Finding an open source community that fits you out of pure interest should not be difficult, you see communities come in all size and shapes. They range from extremely large like the Linux kernel to the very small like your run for the mill script that downloads dubious content from the Internet. Some of the larger projects turn into non-profit organizations to provide a sustainable platform from where they can grow. This is why the Mozilla Foundation was born. You may know Mozilla as the organization who brings you Firefox, a really good Web browser that happens to be licensed as free and open source software.

These larger projects function very much like a typical company, structured into different departments, each with an individual area of operation.

Did you know?

In 1993 a company called Netscape released a version of a graphical Web browser named Mosaic. Mosaic was one of the first of its kind, revolutionary as a matter of fact, quickly becoming the world's most popular browser. In 1995 Microsoft launched Internet Explorer, becoming the first Mosaic opponent. Internet Explorer was bundled with Windows, which gave it quite an advantage. By the end of the 90's Netscape was already in trouble. In order to save the company Netscape leadership created the Mozilla Foundation, and released their browser as open source. They expected that by releasing the code, a community would gather around it, helping the company regain the lost market. They were right!

Once you indentify a potential community with whom you'd like to get involved, you should take some time probing it. Try to figure how a new member's work is contributed. Many projects will make available information on the subject but many others will not. If not in a separate section on their webpage, search forums for instructions on how to get your patches or bugs submitted. Go for frequently asked questions (FAQ) sections and wikis. Sometimes smaller projects include this information into text files that come with the source of the application, check it also! Ultimately if nothing else works search around the Web, maybe one of the community members has posted some information somewhere. A good example of a development guide is at http://ldn.linuxfoundation.org/how-participate-linux-community a Linux kernel developer's guide, which makes also a good read even if you don't plan to take on kernel development.

Suffice is to say that open source has grown from being associated with a software project, to a more extended concept. Open source is now in effect a culture; it symbolizes free sharing of content. Communities gather around the most different concepts of content, written content, video, even hardware such as computer chips or cars are turned into open source.

Wikipedia is one of those communities, even though we must point out that technically it is not considered to be open source. This is because Wikipedia's license does not satisfy the open source principles; it is nonetheless a great example of peer contribution. You may even also come across some projects that have associated literature in the form of a book created gradually by contributors.

5.3 Effective communication

The internet is the foundation of open source communities since users and contributors are scattered across the globe. This also means that communication must be as effective and organized as possible. There must me established infrastructures, and these infrastructures must be well designed and should fully fulfill the community needs and modus operandi.

Though there are no standard communication channels, there is a historic means of communication used extensively across all open source projects: **email**. Email can be used as a great discussion mechanism through **mailing lists**. Mailing lists provide one of the most important community feedback channels, they are filled with developers taking on enthusiastic and technical discussions. There is also a more direct communication approach used extensively across the open source world, **Internet Relay Chat (IRC)** for instantaneous textual communication between two or more parties.

In the past BBSs (Bulletin Board Systems) were used to exchange messages with other users. In 1998 IRCs were introduced, and quickly became the de-facto communication tool for real-time communication throughout the 90's. IRC users have to install a client part of the software to their local computers. Today real-time communication falls in the hands of instant messaging services, but IRCs remain widely used. Due to its open nature it is a good way to meet other people. There are a number of IRC servers you can connect to, but the one used by most open source fellows are the freenode servers (freenode.com). A popular open source IRC is **mIRC** (http://www.mirc.com/).

Web sites are the first and foremost medium for promoting a given open source software and to mass communicate project details. You can host your own Web site with the help of open source Web servers like **Apache** (http://www.apache.org/). If Web hosting is not an area which you or your community has much expertise or interest in, you may consider using a site like **SourceForge**. SourceForge not only provides you with Web hosting, but also offers an integrated platform on which you can run your entire OSS project. For more information visit http://sourceforge.net/. In addition, the last chapter of this book includes a case study using SourceForge.

Did you know?

Part of the ranking process on sourceForge comes from downloads but much of the calculation is driven by community activity. If someone opens a defect, checks in code, makes a feature request, or posts to a forum page; all of that adds to the relative activity score and the project's overall ranking. Projects with a high ranking get noticed.

An example of a successful open source project hosted on SourceForge is the ***Technology Explorer for IBM DB2 (TE)***. TE has over 10,000 downloads with a sourceforge.org ranking consistently in the top 200 projects and in the top ten of over 13,000 database related projects. To find out more about the Technology Explorer for IBM DB2 refer to *Chapter 8*.

A ***Wiki*** is a collaborative Web site developed and maintained by a group of people for knowledge sharing purpose. In an OSS community, active members who are aware of the various technical details of the project, generally publish their own articles in the form of tutorials. http://wiki.rubyonrails.org/ is an example of such a wiki maintained by the Ruby on Rails user group. Wikis are generally open to all for contribution, with restriction only to the deletion of pages.

Feeling the pulse of the community is not restricted to their official communication channels. **Weblogs** (also known as **blogs**) have emerged has one of the most valid means of sharing thoughts around the Web. You are guaranteed to gather a lot of helpful information through blog surfing.

Planets are aggregation of blogs. Every major community has one; it's like a newsletter but much more effective. Developers write about their work and you get to know what's happening from an insider's point of view. It is also one great way members can gain visibility in the community. After all, being credited for great work is the only and best payback community members receive; and everyone likes to be appreciated for their work.

Still the best way by far for feeling your community is to meet the members in person. The most active and passionate members that happen to share the same geographic location organize themselves once in a while, to go out and get to know each other. These are called **user groups**. Look for user groups in your area and go have some fun!. Meetup.com is a Web site where you can easily arrange a local reunion with a specific theme. The Ruby community has done this. Check it out!, maybe there is one for yours already.

Some communities go further and organize worldwide conferences, where they can calmly and effectively discuss the future of the project as well as throw some technical presentations and boost the morale of everyone involved.

There are other types of meetings, however. FOSDEM, short for Free and Open Source Development European Meeting, is a once in a year event in Brussels. It gathers roughly five thousand visitors each year. Developers from all major open source projects come together to exchange knowledge which help promote their community and announce upcoming changes and innovations.

Last but not least, there are events called ***hackatons*** where developers come together and try to implement as many features or close as many bugs as they can. As you can imagine this can provide big pushes into a project's life cycle and as a side effect it improves collaborative working and respect among developers. Now, if you bring a bunch of developers together, give them a beer and a laptop, and you will certainly have a hackaton going as part of the original event.

5.3.1 Communication etiquette and guidelines

As passionate as people are in this kind of projects, it is not uncommon for opinions to diverge and matters to get out of hand. For that reason communities often follow guidelines for user behavior. These guidelines emphasize users' and contributors' communication conventions as to maintain social order and effectiveness.

These are some of the community guidelines:

- Search before you ask.

  This principle is quite simple, sometimes the information you are looking for is clearly stated somewhere within the project documentation or commonly available around the internet. It may be easier to ask on a mailing list or forum but this would be the equivalent to asking someone else to do the work for you.

- Do not engage in acts of verbal aggression.

  The Internet is a wonderful place, it enables people to communicate across continents with no connection whatsoever to their real identity, this triggers some primal human behaviors issues in some of us. Enabling these behaviors in yourself will not bring anything helpful to the conversation, it will however destroy any credibility you have left.

- Do not support acts of verbal aggression.

  If you are taking part on a discussion when all of a sudden a flame war erupts, please do not take part on it, respect other people’s opinions, be constructive and try to positively explain your view. Do remember a hierarchy exists, as in your professional life, you’ll often find people with more authority and lesser technical expertise.

- Know were to ask your questions.

  Usually a quick look around the project Web site is all you need to find your way to a forum or IRC channel where you can ask a specific question. Sometimes in bigger projects these platforms are carefully divided into different and appropriate sections where your question can be answered by someone with appropriate knowledge.

- Show results, you'll earn more respect showing work than talking about it.

  Often when you take part on a technical discussion and your point of view is not getting across, do not stress it, sometimes the best way to do it is by presenting the end result. Even if it turns out it is not the best solution, you will end up helping in

achieving it, hopefully you'll learn something new as well, and you'll have proven yourself to the community.

- Be concise, explain your intentions first.

  Objectiveness is a big plus in any work area, besides saving you and others a lot of time; it simplifies the chore at hand. Across any written medium such as the ones we have seen earlier, it is absolutely essential that you make yourself and your intentions clear in any communication attempt. Start by asking yourself "is this information relevant?"

5.4 Exercises

1. Pick one or more OSS projects mentioned in this book so far, then choose your favorite search engine, and perform the following.

   A. Investigate the project purpose

   B. Find where it is heading, and where it came from

   C. Search for getting started documentation

   Note:

   If you would like to skip this step, Chapter 8 provides an example of an open source project where all the above information is provided.

2. Consider reading the essay "How to ask smart questions" by Eric Raymond on his FAQ page http://www.catb.org/esr/faqs/.

3. Choose the project you enjoyed the most from question one, keep in mind that it must have a forum or IRC channel. Now from what you have read in question two, you are more than able to engage in your community. Go to the community site and join their forum or IRC channel. Participate!

5.5 Summary

This chapter started discussing the motivations for contributing to an open source project. It also described the different roles a new member can take, which are not restricted to coding. The chapter then discussed a range of communication channels used by communities to exchange information; and finally it explained some guidelines and etiquette required when dealing with other members in the community, such as "Do not engage in acts of verbal aggression".

5.6 Review questions

1. How is a project ranked in sourceForge?
2. List some of the duties of a Project Leader.
3. Why is documentation important in an OSS project?

4. Why are there translators in an OSS community?
5. Name two communication etiquette guidelines
6. What are the benefits of open source development?
   A. Delay on bug fixes
   B. Poor security
   C. Size of contribution
   D. All of the above
   E. None of the above
7. What do most open source contributors expect to receive from their contribution?
   A. Money
   B. Recognition
   C. A paid job
   D. All of the above
   E. None of the above
8. Open source contributors should have one of these traits:
   A. Know how to code
   B. Own an IBM Model M keyboard
   C. Quick learner and willing to help others
   D. All of the above
   E. None of the above
9. Which of the following mediums of communication is popular among OSS communities for publishing Web articles or tutorials?
   A. Email
   B. IRC
   C. Internet blog
   D. Wiki
   E. None of the above
10. When engaging in discussions in an OSS project one must
    A. Doubt other people's capabilities
    B. Ignore those who shares different opinions
    C. Prove our point with reason and work

D. All of the above

E. None of the above